WEEK 11 - PYTHON PROGRAMMING

Korean Stock Market Analysis

From scenario to a working web report.

ACT 1 · SCENARIO
define the task

ACT 2 · DESIGN
trace the promise

ACT 3 · TEST
prove the result

ACT 4 · MVP
open the web report

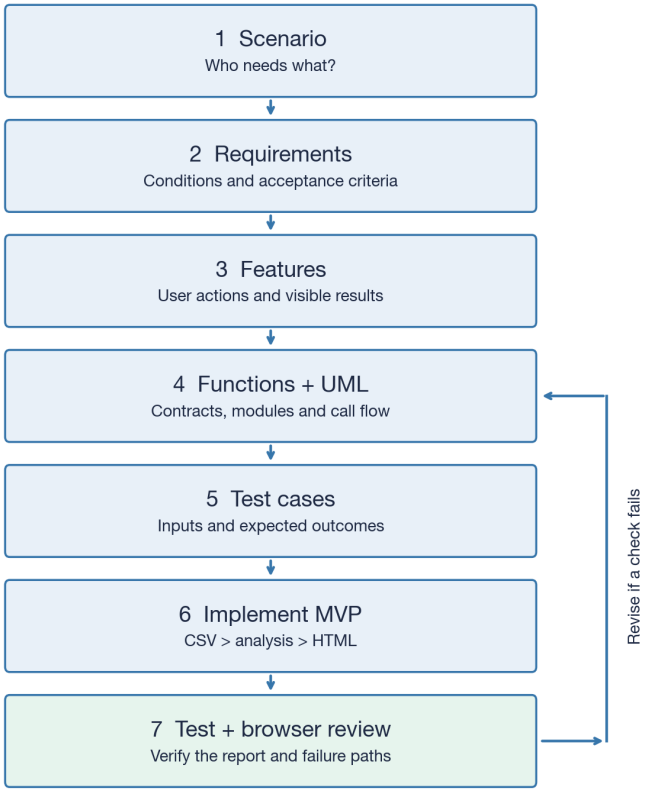
120 minutes, including a break

Instructor: MinGi KYUNG

Development Workflow

Follow one project from the user's problem to a checked web report.

- Scenario document: team-name_user-scenario.md.
- Specification workbook: team-name_spec.xlsx.
- Link SC → REQ → FEAT → FUNC → TEST, then build the MVP.
- Run tests and review the page; revise when a check fails.



Scenario: Minji Reviews the Market

After the market closes, Minji wants to compare a small set of stocks.

- SC-01: prepare a CSV, generate the report and compare markets.
- She runs the report for ALL, then regenerates it for KOSPI.
- She checks the date, daily change ranking and sample summary.
- SC-02: reject bad input. SC-03: generate a valid empty report.

Scope and Data Honesty

A snapshot sample is useful when its limits are visible.

- All supplied prices and volumes are fictional classroom data.
- A single date, two market labels, five example stocks.
- Mean = equal-weight average of the selected sample changes.
- This is not a KOSPI/KOSDAQ index return or a live market feed.

The Input Contract

Define the data before writing the loop.

- UTF-8 CSV; accept an optional BOM; exact ordered header.
- Six-digit stock code stays text: 005930 must keep its zeros.
- Prices: positive whole KRW. Volume: nonnegative whole shares.
- One date; unique codes. Both closes use the same price basis.

OUTPUT

```
date,code,name,market,previous_close,close,volume
2026-11-05,005930,삼성전자,KOSPI,70000,71400,1000000

previous_close: previous trading close (KRW)
close: snapshot closing price (KRW)
volume: shares traded
```

Requirement: Observable Conditions

ACT 1 · REQUIREMENT

Requirement sheet: each REQ links to SC IDs and a check.

- REQ-01: read valid CSV and preserve stock codes.
- REQ-02: reject invalid values, duplicate codes and mixed dates.
- REQ-03: calculate, filter and sort daily percentage changes.
- REQ-04: count up/down/flat and calculate a sample mean.

OUTPUT

```
REQ-03 / Must
Given previous_close=70000 and close=71400,
the daily change is +2%.
For KOSPI, only KOSPI rows appear.
Order: change descending, then code ascending.
```

Requirement: The Web and Failure Rules

ACT 1 · REQUIREMENT

The output and the failure policy are part of the product.

- REQ-05: show date, market, source, units, summary and table in HTML.
- REQ-06: report file/data errors and preserve the old HTML on failure.
- Header-only CSV: valid empty report, count 0 and mean N/A.
- Missing CSV: an error, not a successful empty report.

MVP Cut: Must, Should, Could

ACT 1 · REQUIREMENT

Ship the complete CSV → analysis → HTML path first.

- Must: validated input, analysis, static HTML, clear error handling.
- Should: clickable market filter in the browser.
- Could: API data, history, moving averages, charts and publishing.
- Do not add a new Must until the existing acceptance checks pass.

Feature: Three User Goals

ACT 1 · REQUIREMENT

Features describe what the user can do and observe.

- FEAT-01: load a validated snapshot (REQ-01/02/06).
- FEAT-02: compare stocks by market (REQ-02/03/04).
- FEAT-03: generate and view a web report (REQ-05/06).
- Workbook Feature sheet: link FEAT IDs to Related requirement IDs.

SHELL

```
python3 main.py --input data/prices.csv
python3 main.py --market KOSPI
python3 main.py --output docs/index.html
```

Feature Sheet: Market Comparison

ACT 1 · REQUIREMENT

Write normal, empty and error outcomes before implementing.

- Usage: python3 main.py --market KOSPI.
- Flow: validate snapshot → filter → calculate → sort → summarize.
- Result: 3 stocks; 005930, 035420, 000660; mean +0.33%.
- Empty selection: 0 stocks and N/A. Unknown market: CLI error.

OUTPUT

```
FEAT-02 / basic
serves: REQ-02, REQ-03, REQ-04
ALL:    5 stocks, up 2 / down 2 / flat 1
        sample mean +0.40%
KOSPI:  3 stocks, sample mean +0.33%
KOSDAQ: 2 stocks, sample mean +0.50%
```

Function: Contracts Before Bodies

Keep values and failure rules consistent across module boundaries.

- FUNC-01 parse_row(row): strings → typed quote; ValueError on invalid fields.
- FUNC-02 load_quotes(path): CSV → list; missing file raises OSError.
- FUNC-04 analyze(quotes, market): validated list → fresh sorted rows.
- FUNC-05 summarize(rows): counts and mean; empty mean is None.

OUTPUT

```
Quote = {date, code, name, market,
         previous_close: int, close: int,
         volume: int}
Analyzed row = Quote + {change_pct: float}
Summary = {count, up, down, flat, mean}
```

Implement the Smallest Rule

ACT 2 · FUNCTION

Compute with full precision. Round only in the report.

- Formula: (close previous_close) / previous_close × 100.
- Reject a zero or negative price before dividing.
- A good case, a fall and a flat price are separate checks.

PYTHON

```
def change_pct(previous_close, close):
    if previous_close <= 0 or close <= 0:
        raise ValueError("prices must be positive")
    difference = close - previous_close
    return difference / previous_close * 100

assert change_pct(70000, 71400) == 2.0
assert change_pct(200000, 198000) == -1.0
assert change_pct(100, 100) == 0.0
print("Daily change checks passed.")
```

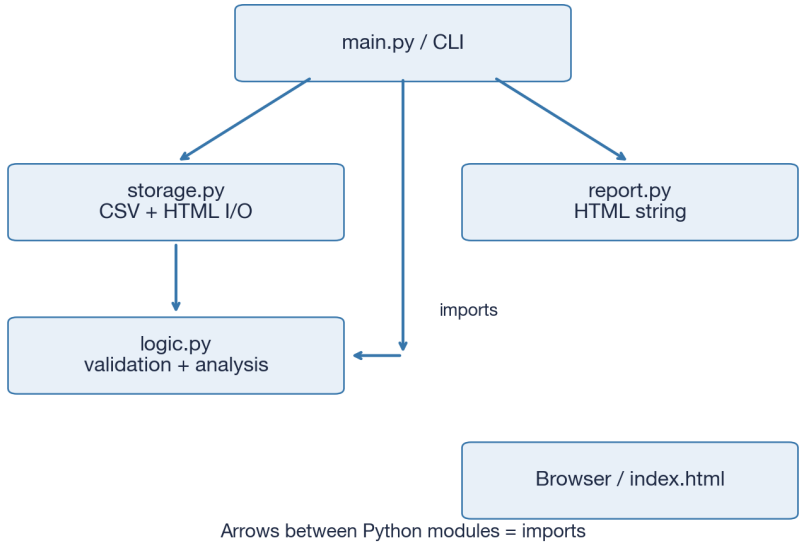
OUTPUT

```
Daily change checks passed.
```

Modules: Keep I/O at the Edges

The module diagram tells us where each responsibility lives.

- main: arguments, orchestration, messages and exit codes.
- storage: CSV reads and atomic HTML writes; imports parse_row.
- logic: validation and analysis. report: escaped HTML strings.
- Neither logic nor report opens files or asks the user questions.



Test: Give Each Promise Evidence

Use fixed examples, not a changing online price.

- TEST-01/12: CSV schema, BOM, empty file and leading-zero codes.
- TEST-02/04: exact changes, market filter and tie-breaking.
- TEST-03/07: invalid fields, duplicate codes and mixed dates.
- TEST-05/06: sample summary, empty selection and N/A.

SHELL

```
cd 11week/examples/korea_market_mvp
python3 -m unittest -v
```

Test the Web and the Failure Boundary

ACT 2 · FUNCTION

Correct arithmetic is only part of a working report.

- TEST-08: escape a stock name containing a script tag.
- TEST-09: missing CSV or bad header leaves old HTML unchanged.
- TEST-10: simulated output replacement failure preserves the file.
- TEST-11: main generates the expected KOSPI HTML.

OUTPUT

```
Workbook Test sheet:
TEST-01 to TEST-12: automated, PASS
TEST-13: browser visual check, Not run
Record Actual result separately from Expected.

Use TemporaryDirectory for file tests.
Do not overwrite the class demo in a test.
```

UML: Actor, Goals and Call Order

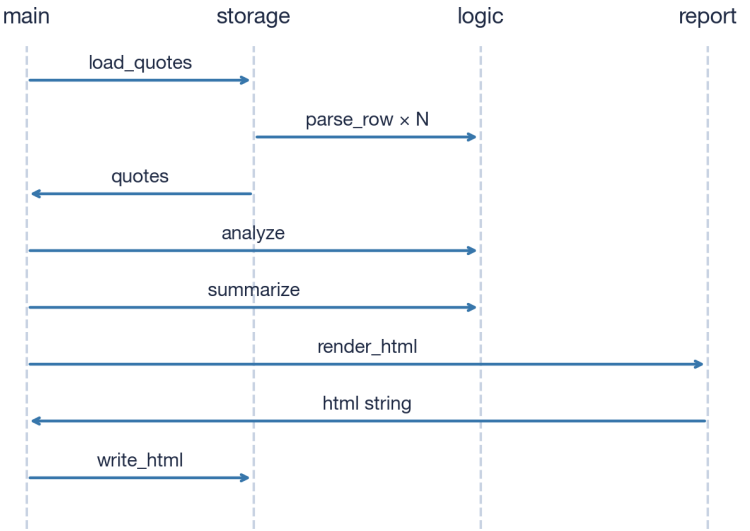
Choose the diagram that answers the reader's question.

- Use case: Student → load snapshot, compare stocks, view report.
- Component: which module depends on which.
- Sequence: the calls made during one successful run.
- Activity: the normal path and the stop-on-error branches.

Sequence: One Successful Run

Every call name appears in the function contract table.

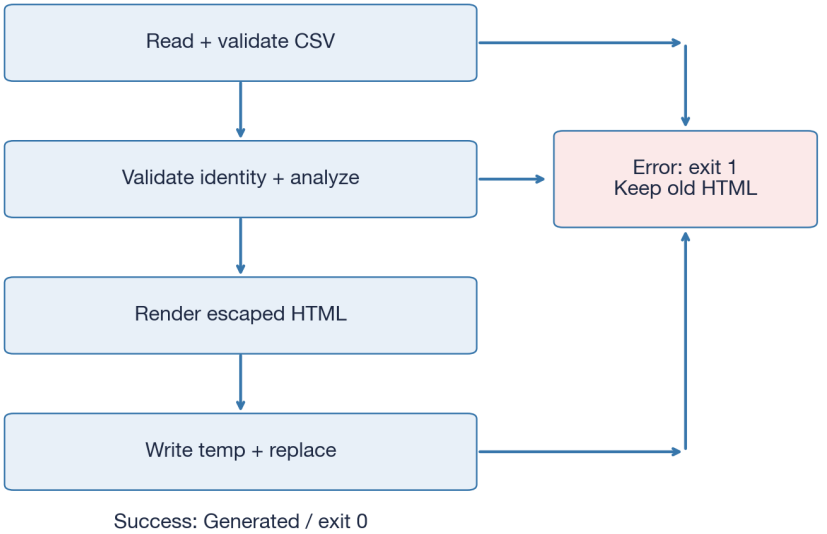
- main → load_quotes → parse_row for each row.
- main → analyze → summarize → render_html.
- main → write_html; success is reported only after replacement.
- Browser opens the generated file after Python finishes.



Activity: Stop Before Replacing Output

Validate first, then render, then commit the output file.

- Bad input ends with a reason and exit code 1.
- A complete new file replaces the old file in one step.
- A valid empty input follows the successful report path.
- The full Mermaid diagrams are included in project-spec.md.



Trace One Requirement End to End

ACT 3 · UML

The same ID must survive all the way to a passing check.

- REQ-03: daily change and market ranking.
- FEAT-02: compare stocks by market.
- FUNC-03 change_pct; FUNC-04 analyze: code contracts.
- Sequence: analyze call. TEST-02/04: numeric and ordering proof.

OUTPUT

```
REQ-06 -> FEAT-01 / FEAT-03
       -> load_quotes / write_html / main
       -> activity error branch
       -> TEST-09 / TEST-10

Practice: trace REQ-05 with a neighbour.
```

BREAK

Break

Ten minutes. After the break: build and inspect the MVP.

- Keep project-spec.md and the example directory open.
- Next: fake a full pipeline, replace the fakes, open the page.

Prototype a Complete Path

ACT 4 · MVP

A fake makes the pipeline runnable before every rule exists.

- Start with a known quote and a tiny HTML table.
- Keep the planned names and return types.
- Replace one fake at a time: rules, CSV, HTML, output.
- Final MVP: no fake on any Must-feature path.

PYTHON

```
def load_quotes(path):
    return [{"code": "005930", "change_pct": 2.0}]

def render_html(rows):
    return "<h1>FAKE classroom report</h1>"

rows = load_quotes("data/prices.csv")
print(render_html(rows))
```

OUTPUT

```
<h1>FAKE classroom report</h1>
```

CSV to Validated Quotes

Convert text once, and keep the code as a string.

- csv.DictReader reads fields as strings.
- Check the exact header before trusting any row.
- parse_row validates and converts prices and volume.
- Validate every row, even one outside the selected market.

PYTHON

```
import csv, io
source = ("code,previous_close,close\n"
          "005930,70000,71400\n")
row = next(csv.DictReader(io.StringIO(source)))
print(row["code"], type(row["code"]).__name__)
difference = (int(row["close"])
              - int(row["previous_close"]))
print(difference)
```

OUTPUT

```
005930 str
1400
```

Filter, Sort and Summarize

Keep calculations independent of HTML.

- Reject mixed dates and duplicate codes before filtering.
- Return new dictionaries; leave the loaded quotes unchanged.
- Sort on unrounded change, then stock code for equal values.
- Empty rows give mean None; the view turns that into N/A.

OUTPUT

```
ALL order from the supplied fictional CSV:
247540   +3.00%
005930   +2.00%
035420   +0.00%
000660   -1.00%
035900   -2.00%
mean     +0.40%  (sample, equal weight)
```

HTML Is a View of the Result

Formatting changes presentation; it must not change the calculation.

- render_html returns a string; storage writes it.
- Use html.escape for stock names and other external text.
- Show sign and number as well as color.
- Include date, source, fictional-data label and units.

PYTHON

```
from html import escape
name = "<b>Example</b>"
change = 2.0
print(escape(name))
print(f"{change:+.2f}%")
```

OUTPUT

```
&lt;b&gt;Example&lt;/b&gt;
+2.00%
```

main: Assemble the Program

The top-level function coordinates the contracts.

- argparse: --input, --output, --market.
- Catch data/file errors at the user-facing boundary.
- Prepare all results before write_html is called.
- Exit 0 on success, 1 on data/I/O failure, 2 on CLI misuse.

OUTPUT

```
quotes = load_quotes(args.input)
rows = analyze(quotes, args.market)
summary = summarize(rows)
html = render_html(rows, summary, market, day)
write_html(args.output, html)

Only now print Generated ...
```

Run the MVP and Open the Page

No package installation or network data is required.

- Run from examples/korea_market_mvp/.
- The default report contains five fictional stock rows.
- Open docs/index.html directly, or serve docs locally.
- The server serves saved HTML; regenerate to change the market.

SHELL

```
python3 -m unittest -v
python3 main.py
python3 -m http.server 8000 \
  --directory docs --bind 127.0.0.1
# Open http://localhost:8000
# Ctrl+C stops the local server.
```

Refactor with Evidence

Keep behavior identical while improving the structure.

- Before and after each move: run all 12 checks.
- Rename vague variables to quote, rows and summary.
- Keep FIELDS and MARKETS in one place.
- Extract HTML formatting; preserve numbers, order and messages.

OUTPUT

```
Feature commit: add KOSPI filtering
Refactor commit: extract render_html
Fix commit: preserve leading zero in stock code

These are different changes and need different proof.
```

Lab: Build Your Own Slice

ACT 4 · MVP

18 minutes: one branch, one visible result, one reviewable PR.

- 4 min: trace SC → REQ → FEAT → FUNC → TEST in the attachments.
- 9 min: implement a slice and replace its fake.
- 5 min: run checks, inspect HTML, commit and open a PR.
- Files: labs/session2/team-name_spec.xlsx and team-name_user-scenario.md.

Acceptance Demo and Submission

ACT 4 · MVP

A working web report is the evidence for the documents.

- Show ALL, then KOSPI; explain why their sample means differ.
- Show invalid input preserving the previous HTML.
- Submit the filled workbook and scenario .md with code, HTML and PR link.
- Record Actual result / Pass status; attach a browser screenshot.

Exit Check

Explain one complete chain without changing its vocabulary.

- Why must 005930 stay a string?
- Why is the sample mean not an index return?
- What is the difference between an empty input and a missing file?
- Where is rounding done? Where is HTML escaping done?
- Which tests prove the previous web report survives failure?